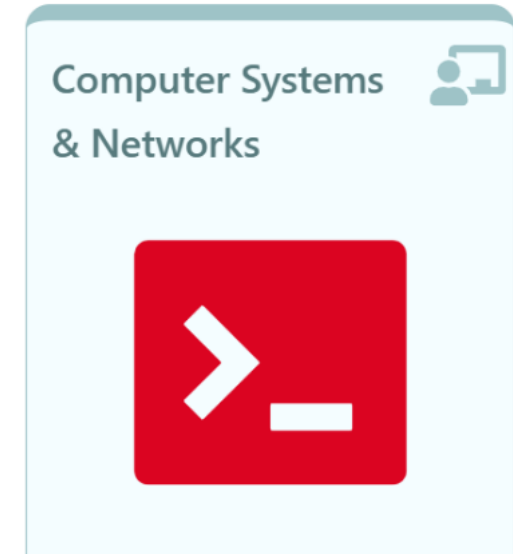
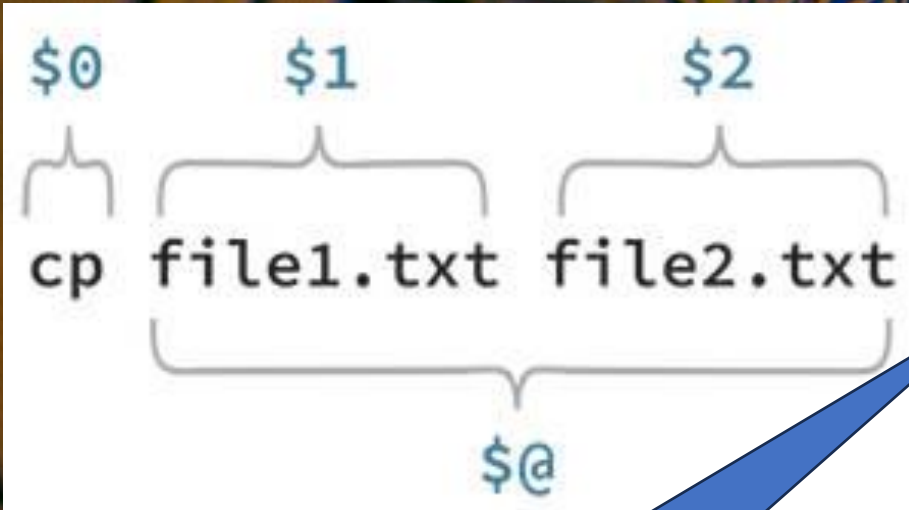


Computer Systems & Networks



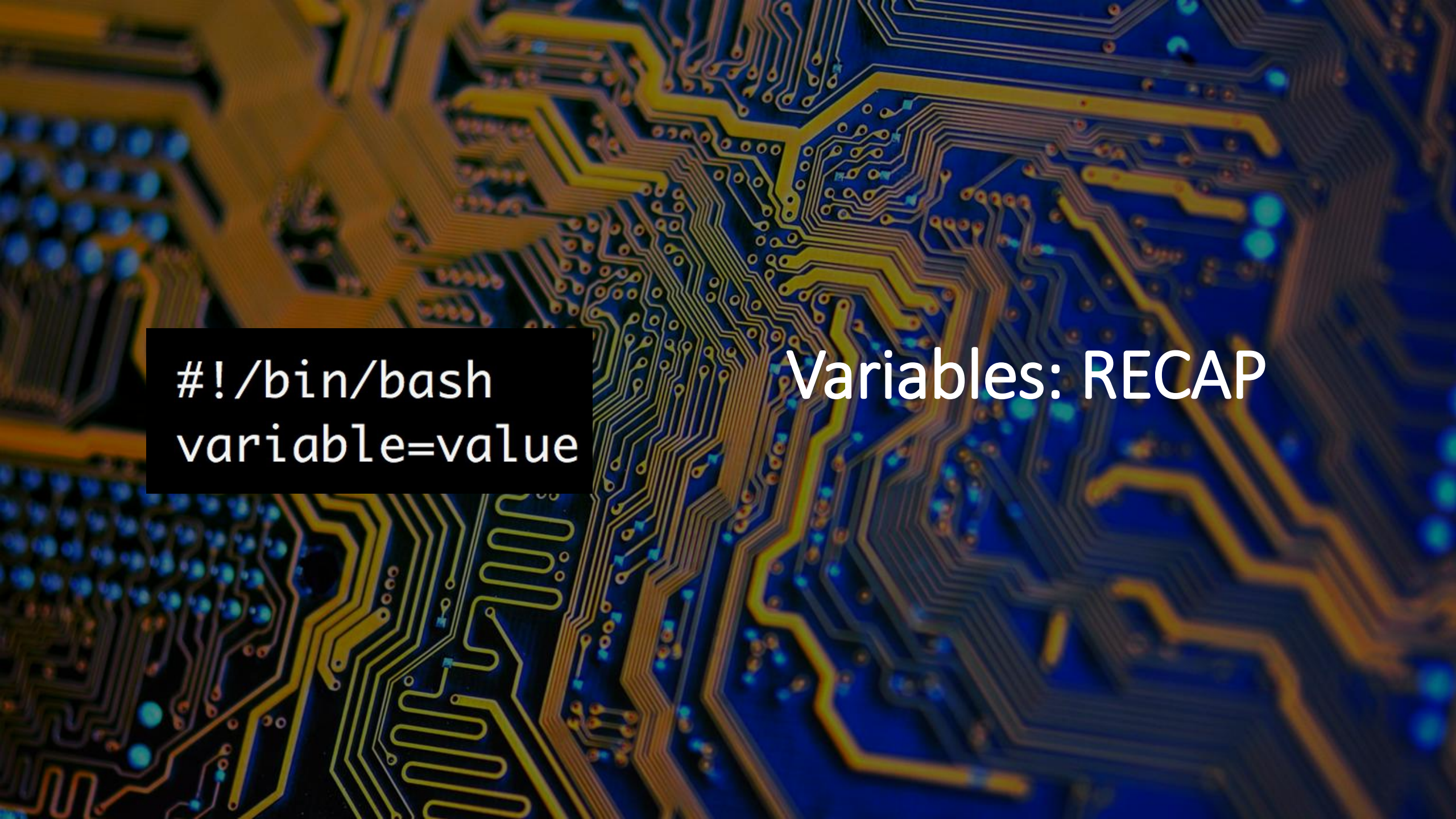


Positional Parameters

Exit codes

Positional parameters are **special variables** in shell scripting that store **arguments passed to a script or function**, based on their **position**

An **exit code** is a **number returned by a process or command** when it finishes running (tells system whether the *command succeeded or failed*)



```
#!/bin/bash  
variable=value
```

Variables: RECAP

Variables

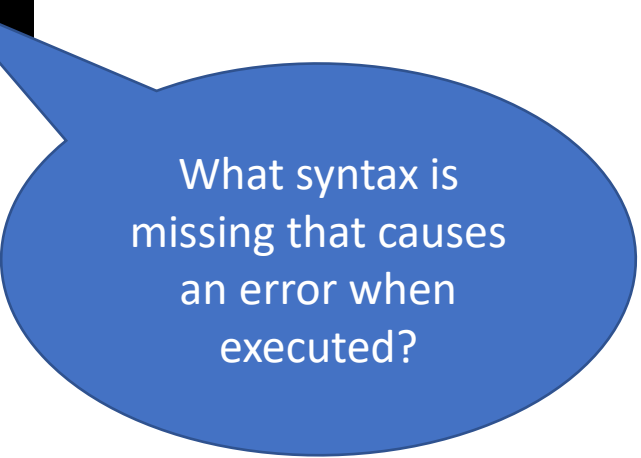
- Variables are a key part of any programming language.
- A very simple use of variables is shown here:

```
echo Enter your first number:
read first
echo Enter your second number:
read second
z=`expr $first + $second`
echo The sum is ${z}
```

Variables

- Get input from the user of the script and assign that input to a variable using the `read` command:

```
echo -n "What is your name?"  
read name  
echo "Hello ${name}! Nice of you to join us"
```



What syntax is missing that causes an error when executed?

Variables: Summary

- Don't need to declare

```
caroline@caroline-VirtualBox:~$ x=10
caroline@caroline-VirtualBox:~$ echo $x
```

- read command

```
caroline@caroline-VirtualBox:~$ read number
45
caroline@caroline-VirtualBox:~$ echo $number
```

- No type

```
caroline@caroline-VirtualBox:~$ x=8
caroline@caroline-VirtualBox:~$ y=5
caroline@caroline-VirtualBox:~$ z=$x+$y
caroline@caroline-VirtualBox:~$ echo $z
```

The output is:

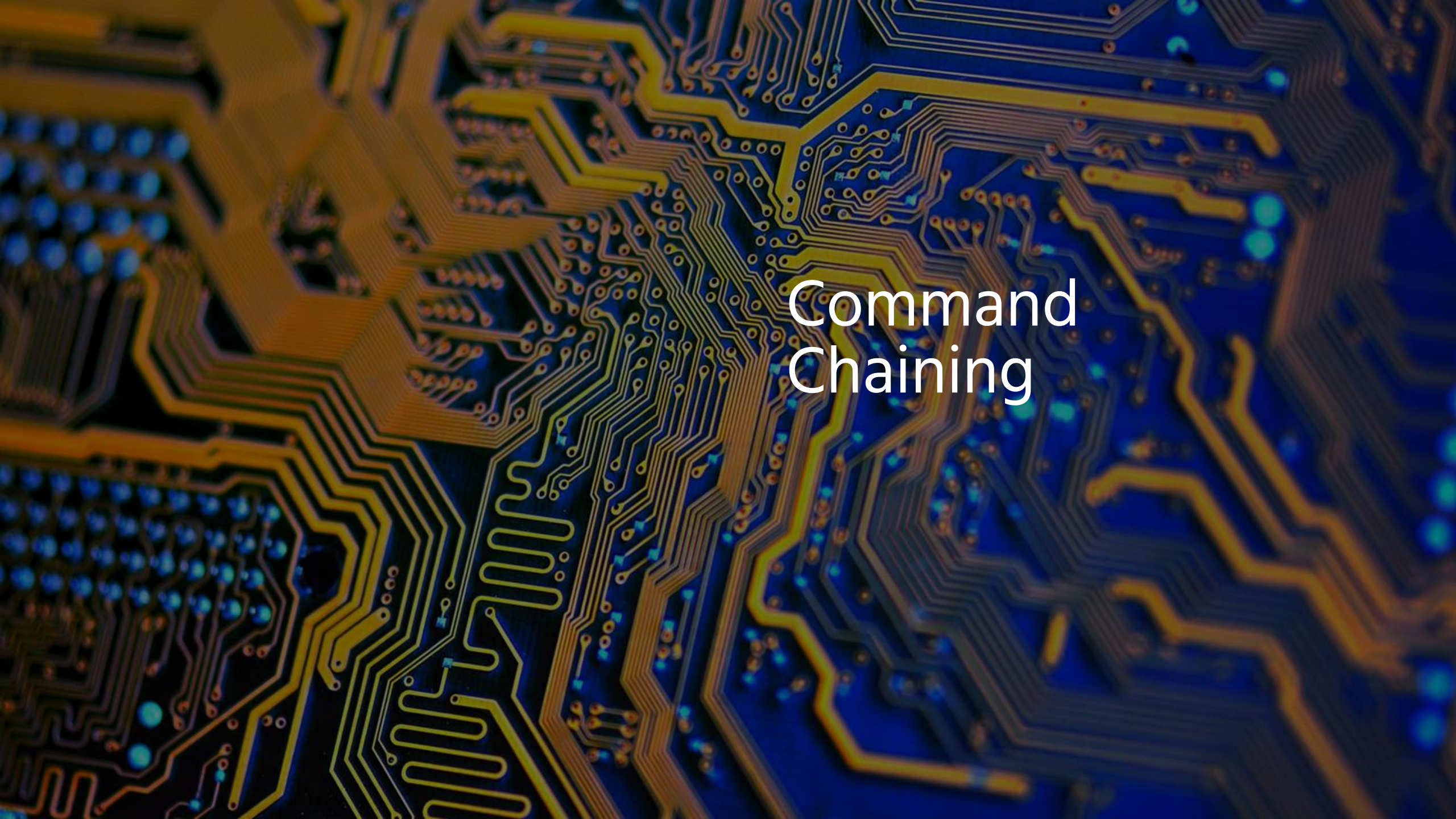
- expr

```
x=8
y=5
z=$x+$y
echo $z
```

```
expr $x+$y
```

```
z=`expr $x + $y`
echo The sum is $z
```

Now try out
multiplying 6 by 4



Command Chaining

Command Chaining ; && ||

;(semicolon) → run one command right after another

```
$ls ; pwd ; whoami
```

With ; the second command will run even if the first one fails

&& (and) → second command to only run if the first command is successful

```
$mkdir compsys2026 && cd compsys2026
```


Command Chaining ; && ||

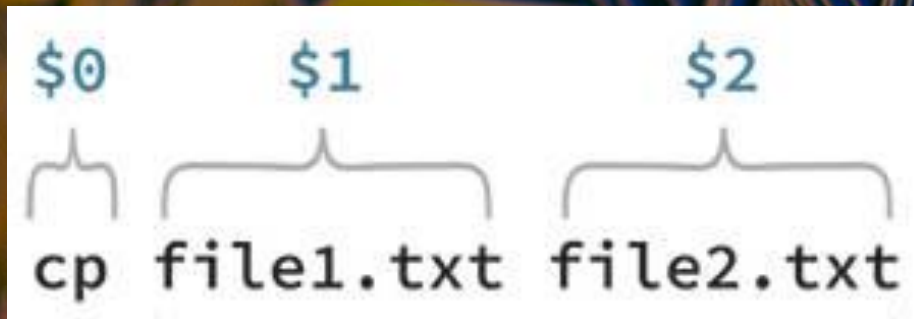
Check if a directory exists [-d "/path/dir/"]

```
compsys@compsys-virtualbox:~$ [ -d "temp" ] && echo "Directory exists"
Directory exists
compsys@compsys-virtualbox:~$ [ ! -d "rainy" ] && echo "Directory DOESN'T exists"
Directory DOESN'T exists
```

|| (or) → Here, you want to execute second command only if first command ***is not*** successful

```
$ [ -d "newHDipFolder" ] && echo "Directory exists" || mkdir ~/newHDipFolder
```

- b filename - Block special file
- c filename - Special character file
- d directoryname - Check for directory Existence
- e filename - Check for file existence, regardless of type (node, directory, socket, etc.)
- f filename - Check for regular file existence not a directory
- G filename - Check if file exists and is owned by effective group ID
- G filename set-group-id - True if file exists and is set-group-id
- k filename - Sticky bit
- L filename - Symbolic link
- O filename - True if file exists and is owned by the effective user id
- r filename - Check if file is a readable
- S filename - Check if file is socket
- s filename - Check if file is nonzero size
- u filename - Check if file set-user-id bit is set
- w filename - Check if file is writable
- x filename - Check if file is executable



```
$0      $1      $2
cp  file1.txt file2.txt
```

Positional Parameters in Bash

Positional parameters are **special variables** in shell scripting that store **arguments passed to a script or function**, based on their **position**

Variables → Positional Parameters

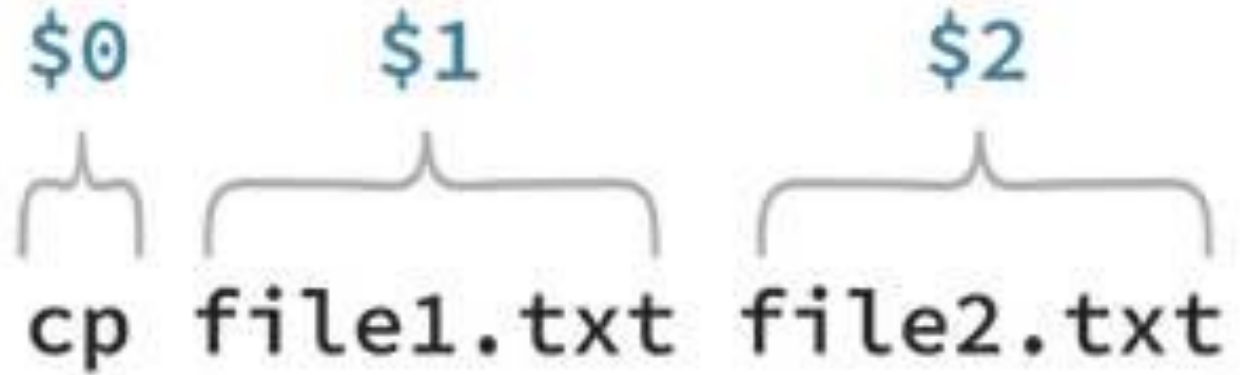
\$0 \$1 \$2
cp file1.txt file2.txt

- The **\$0** variable contains the name of the script itself. Additionally, you can pass *arguments* to your script:

```
compsys@compsys-virtualbox:~$ cat pospa*  
#!/bin/bash  
echo -n "What's your name? "  
read name  
echo "Hello ${name}! You're looking at your file $0"
```

A dollar sign followed by a number *N* corresponds to the **Nth** argument passed to the script

Positional Parameters: \$



...referenced using the \$ sign.

\$0 is the **script name** or else the **command** given

\$1 variable is used to read the **first** command line argument

\$2 variable is used to read the **second** command line argument etc.

(Two digits are written as **\${10}**, **\${11}** and so on.)

\$0 \$1 \$2
cp file1.txt file2.txt

Positional Parameters: **\$#**

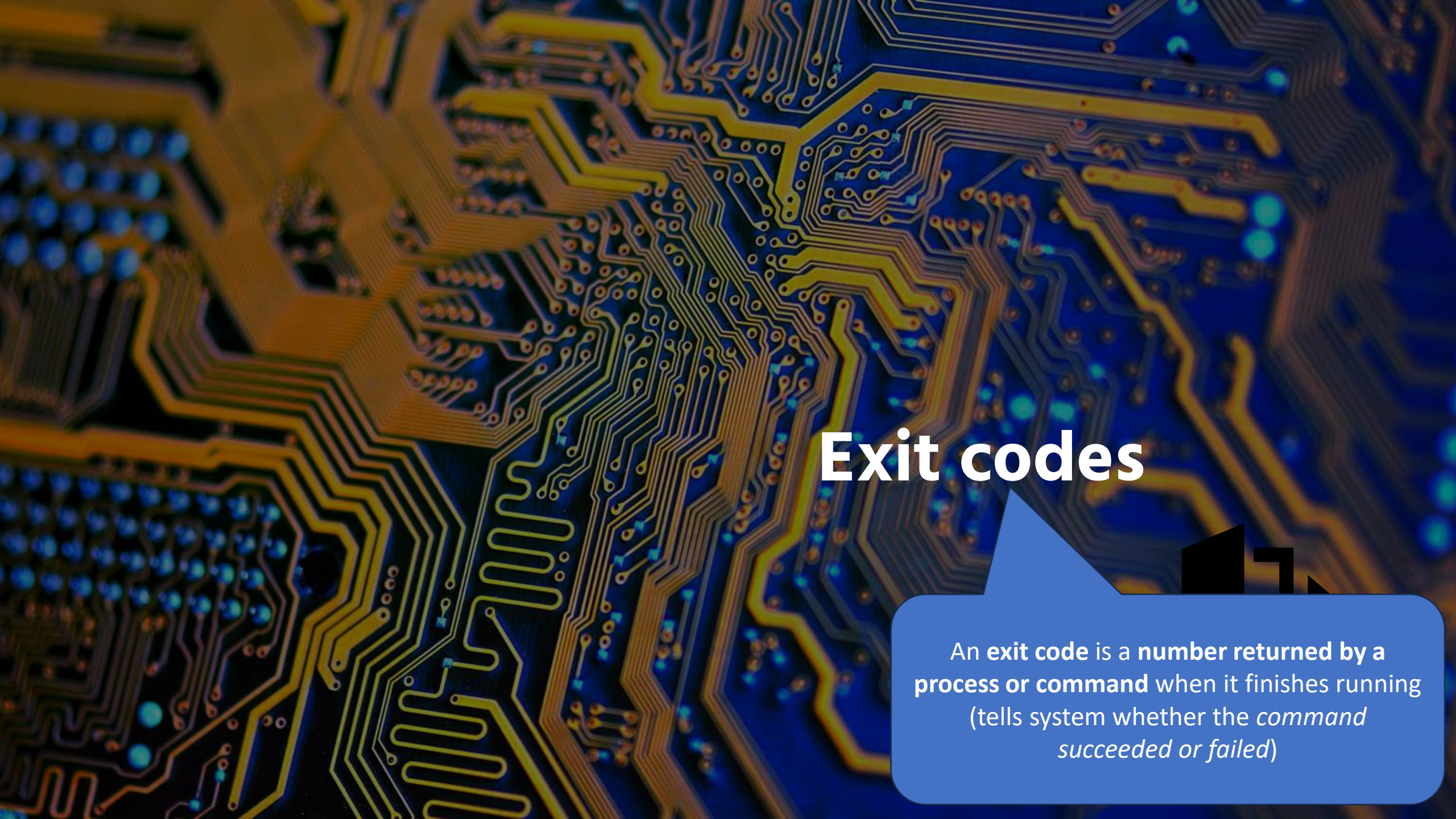
\$# the number of positional parameters, excluding **\$0**.

\$# is often used to check that the user has entered the correct number of command line arguments were passed to the script.

\$0 \$1 \$2
cp file1.txt file2.txt

Positional Parameters: **\$***

\$* references *all* the arguments passed to a program
(useful in programs that take an indefinite or variable number of arguments)



Exit codes

An **exit code** is a **number returned by a process or command** when it finishes running (tells system whether the *command succeeded or failed*)

Exit Codes

indicates the response from the command or a script after execution

- help us determine whether a process ran Successfully

For the bash shell's purposes, a command which exits with a:

- zero (0) exit status = **success**;
- non-zero (1-255) exit status = **failure**

command causes the shell or program to terminate

- You can set the exit code of your own script with the **exit** command.

\$?



The special variable **\$?** contains a number that indicates the **exit code (=result) of the last command**



0 →  Success



Non-zero →  **Failure or error** (A value of 1 generically indicates that some error has happened)



You can check the exit code of the **last executed command** using: **\$echo \$?**

```
compsys@compsys-virtualBox:~$ cat employee.txt
compsys@compsys-virtualBox:~$ echo $?
0
compsys@compsys-virtualBox:~$ grep -q asdfjkl; /etc/passwd
compsys@compsys-virtualBox:~$ echo $?
1
```

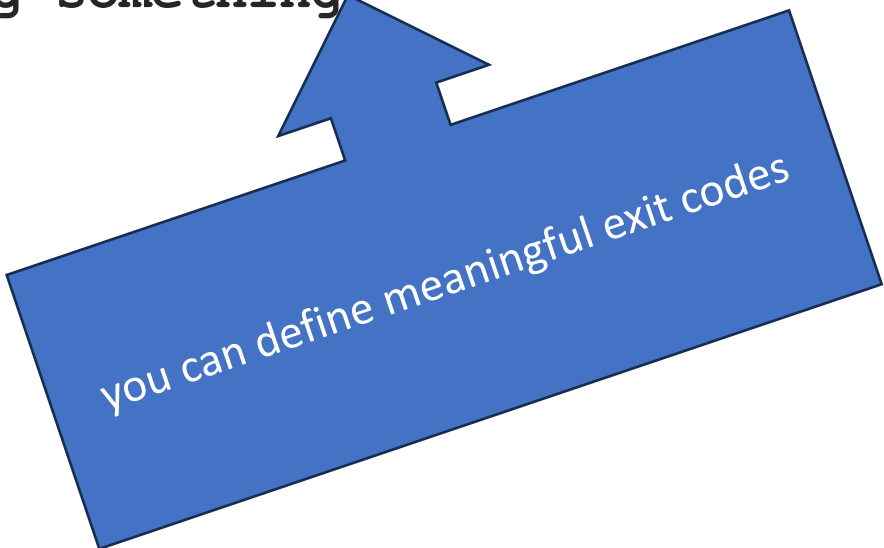
Use **\$?** variable to see if the previous command completed successfully (success=0)


```
#!/bin/bash

#Quote variables/filenames to prevent problems if filename has
spaces

if [ ! -f "compsys.txt" ]; then
    echo "The file does not exist. Displaying an error message."
    exit 23  # Random code for error condition
fi

echo "The file exists, and we will do something else."
echo "Here you could have your script doing something "
exit 0  # Success
```



you can define meaningful exit codes

```
#!/bin/bash
```

```
E_FILE_NOT_FOUND=3
```

```
if [ ! -f "compsys.txt" ]; then
```

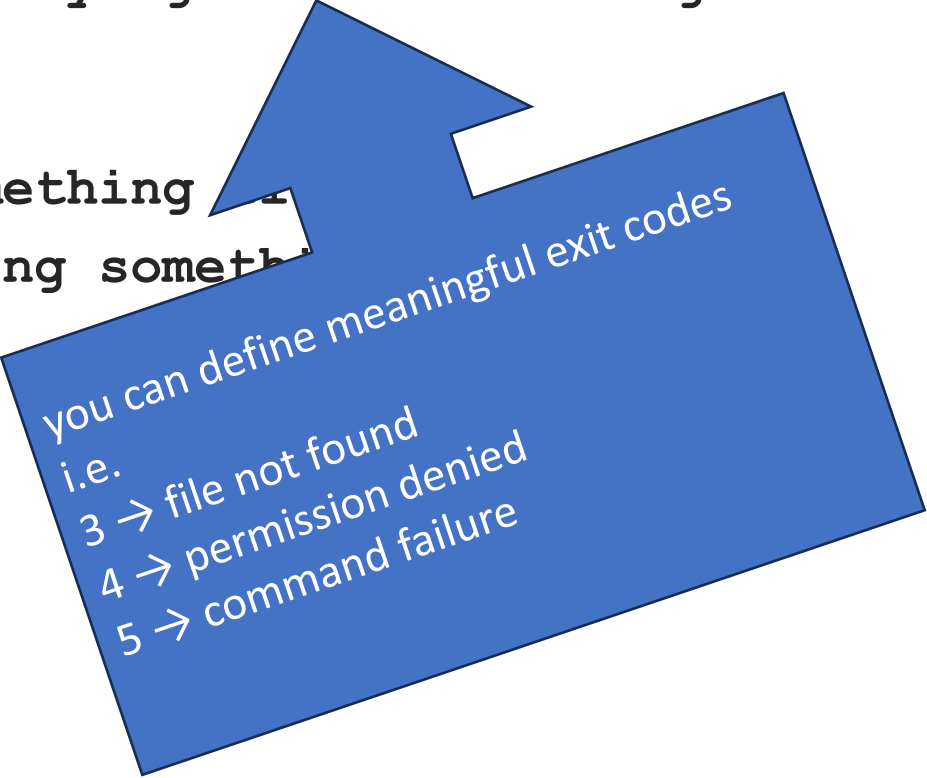
```
    echo "The file does not exist. Displaying an error message."  
    exit $E_FILE_NOT_FOUND
```

```
fi
```

```
echo "The file exists, and we will do something"
```

```
echo "Here you could have your script doing something"
```

```
exit 0 # Success
```



you can define meaningful exit codes
i.e.
3 → file not found
4 → permission denied
5 → command failure

```
$/myexit_test.sh
```

The file does not exist. Displaying an error message.

```
$echo $?
```

```
_____ #what gets returned?
```

```
$touch compsys.txt
```

```
$/myexit_test.sh
```

The file exists, and we will do something else.

Here you could have your script doing something.

```
$echo $?
```

```
_____ #what gets returned?
```



```
#!/bin/bash
# you should always include a comment to describe the workings of a script
# Description: Check if a file exists; create it if it doesn't, send errors to /dev/null

FILE="compsys.txt"
if [ ! -f "$FILE" ]; then          # Check if the file exists
    echo "File '$FILE' does not exist. Creating it..."
    #Attempt to create the file, redirecting errors to /dev/null
    touch "$FILE" 2>/dev/null
    if [ $? -ne 0 ]; then # Check if touch succeeded
        echo "Error: Failed to create '$FILE'."
        exit 1  # Exit with error code
    else
        echo "File '$FILE' created successfully."
    fi
else
    echo "File '$FILE' already exists. No action needed."
fi
exit 0
```

PRACTICE: Using positional parameters & redirection to append to fields in file

- Write a script that can be used to add in a new employees details to the **employee.txt** file

```
Caroline@ICTSkills>~$ cat employee.txt
100 frank manager sales €5000
200 caroline developer technology €5500
300 mary sysadmin technology €7000
400 rosanne manager marketing €9500
500 eamonn dba technology €6000
```

- What could be the **steps** you might take with this?

Try out today's lab

01: Search and Filter



sort · cut · grep

02: Regular Expressions



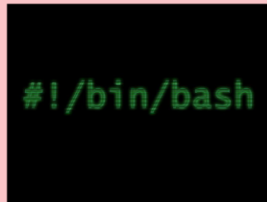
basic RegEx · extended RegEx · metacharacters

03: Positional Parameters



Positional Parameters · exit

Lab



grep · RegEx · Variables · Positional Parameters · Exit

Manipulate Files & Contents

01. sort, cut

02. grep

03. Variables

04. Positional Parameters

Cisco NetAcad

- `$*` references all the arguments passed to a program

```
#!/bin/bash
echo "There were $# variables passed "
echo "All arguments passed can be viewed using the \${*} variable E.G. ${*} "
```

EXERCISE: Practice your coding...

Q: what is `let`? Try `man let`; do you need to use `help let`?

```
#!/bin/bash

echo -e "\$1=$1"
echo -e "\$2=$2"

let add=$1+$2
let sub=$1-$2
```